# PATIENT TRIAGE & APPOINTMENT SYSTEM

C++ Object-Oriented Programming Project

| | |
|---|---|
| **Project** | Patient Triage & Appointment System |
| **Language** | C++ |
| **Source file** | oop-final code (1).cpp |
| **Core approach** | Object-Oriented Programming |

Project source code and concise technical overview

## 1. Project Overview

The Patient Triage & Appointment System is a console-based C++ program that maintains a waiting queue and selects patients according to their priority level. Users can add patients, view the waiting queue, call the next patient, and exit the program.

## 2. Patient Categories and Priority

| Patient category | Assigned priority |
|---|---|
| General Patient | 1 |
| Emergency Patient | 3 |
| Critical Patient | 5 |

## 3. Object-Oriented Programming Concepts

**Abstraction:** Patient is an abstract base class and declares getPatientType() as a pure virtual function.

**Inheritance:** GeneralPatient, EmergencyPatient, and CriticalPatient inherit from Patient.

**Polymorphism:** The program stores derived objects through Patient pointers and calls virtual functions.

**Encapsulation:** Patient data members are protected and are accessed through class methods where appropriate.

**Exception handling:** A try/catch block handles an invalid patient category selection.

**Dynamic memory:** Patient objects are created with new and released with delete.

## 4. Main Program Operations

• Add Patient — collects patient details and creates the selected patient type.

• View Waiting Queue — displays patients in their current priority order.

• Call Next Patient — displays and removes the first patient from the queue.

• Exit — closes the program and releases any patients still in the queue.

Queue ordering: patients with a higher severity value are placed first. When two patients have the same severity, the earlier arrival is placed first.

## 5. Complete C++ Source Code

The following listing reproduces the uploaded source file. Line numbers are included for readability; the code content is preserved.

```
001  #include <iostream>
002  #include <string>
003  using namespace std;
004  // ABSTRACT BASE CLASS (ABSTRACTION)
005  class Patient
006  {
007  protected:
008      int patientID;
009      string name;
010      int age;
011      string medicalCondition;
012      int severity;
013      int arrivalOrder;
014  public:
015      // Constructor
016      Patient(int id, string n, int a,
017              string condition, int sev, int arrival)
018      {
019          patientID = id;
020          name = n;
021          age = a;
022          medicalCondition = condition;
023          severity = sev;
024          arrivalOrder = arrival;
025      }
026      // Pure Virtual Function
027      virtual string getPatientType() = 0;
028      // Virtual Function (Polymorphism)
029      virtual void display()
030      {
031          cout << "\nPatient ID      : " << patientID;
032          cout << "\nName            : " << name;
033          cout << "\nAge             : " << age;
034          cout << "\nCondition       : " << medicalCondition;
035          cout << "\nPriority Level  : " << severity;
036          cout << "\nArrival Order   : " << arrivalOrder;
037      }
038      int getSeverity()
039      {
040          return severity;
041      }
042      int getArrivalOrder()
043      {
044          return arrivalOrder;
045      }
046      virtual ~Patient() {}
047  };
048  // GENERAL PATIENT CLASS (INHERITANCE)
049  // Automatically gets priority 1
050  class GeneralPatient : public Patient
051  {
052  public:
053      GeneralPatient(int id, string n, int a,
054                     string condition, int arrival)
055
056          : Patient(id, n, a, condition, 1, arrival)
057      {
058      }
059      string getPatientType()
060      {
061          return "General Patient";
062      }
063  };
064  // EMERGENCY PATIENT CLASS (INHERITANCE)
065  // Automatically gets priority 3
066  class EmergencyPatient : public Patient
067  {
068  public:
069      EmergencyPatient(int id, string n, int a,
070                       string condition, int arrival)
071
072          : Patient(id, n, a, condition, 3, arrival)
073      {
074      }
075      string getPatientType()
076      {
077          return "Emergency Patient";
078      }
079  };
080  // CRITICAL PATIENT CLASS (INHERITANCE)
081  // Automatically gets priority 5
082  class CriticalPatient : public Patient
083  {
```

```
public:
    CriticalPatient(int id, string n, int a,
                    string condition, int arrival)

        : Patient(id, n, a, condition, 5, arrival)
    {
    }
    string getPatientType()
    {
        return "Critical Patient";
    }
};
// MAIN FUNCTION
int main()
{
    // Array of patient pointers
    Patient* queue[100];
    int count = 0;
    int arrivalCounter = 1;
    int choice;
    do
    {
        cout << "\n====================================";
        cout << "\n PATIENT TRIAGE & APPOINTMENT SYSTEM";
        cout << "\n====================================";
        cout << "\n1. Add Patient";
        cout << "\n2. View Waiting Queue";
        cout << "\n3. Call Next Patient";
        cout << "\n4. Exit";
        cout << "\n\nEnter Choice: ";
        cin >> choice;
        switch(choice)
        {
        // ADD PATIENT
        case 1:
        {
            try
            {
                int id;
                int age;
                int type;
                string name;
                string condition;
                cout << "\nEnter Patient ID: ";
                cin >> id;
                cin.ignore();
                cout << "Enter Patient Name: ";
                getline(cin, name);
                cout << "Enter Age: ";
                cin >> age;
                cin.ignore();
                cout << "Enter Medical Condition: ";
                getline(cin, condition);
                cout << "\nSelect Patient Type";
                cout << "\n1. General Patient";
                cout << "\n2. Emergency Patient";
                cout << "\n3. Critical Patient";
                cout << "\nEnter Type: ";
                cin >> type;
                // Exception Handling
                if(type < 1 || type > 3)
                {
                    throw type;
                }
                // Polymorphism
                if(type == 1)
                {
                    queue[count] =
                    new GeneralPatient(
                    id,name,age,
                    condition,arrivalCounter);
                }
                else if(type == 2)
                {
                    queue[count] =
                    new EmergencyPatient(
                    id,name,age,
                    condition,arrivalCounter);
                }
                else
                {
                    queue[count] =
                    new CriticalPatient(
                    id,name,age,
                    condition,arrivalCounter);
                }
                count++;
                arrivalCounter++;
                // MANUAL SORTING
```

```
                // Highest Priority First
                for(int i=0;i<count-1;i++)
                {
                    for(int j=i+1;j<count;j++)
                    {
                        bool swapNeeded = false;
                        if(queue[j]->getSeverity() >
                           queue[i]->getSeverity())
                        {
                            swapNeeded = true;
                        }
                        else if(queue[j]->getSeverity() ==
                                queue[i]->getSeverity())
                        {
                            if(queue[j]->getArrivalOrder() <
                               queue[i]->getArrivalOrder())
                            {
                                swapNeeded = true;
                            }
                        }
                        if(swapNeeded)
                        {
                            Patient* temp = queue[i];
                            queue[i] = queue[j];
                            queue[j] = temp;
                        }
                    }
                }
                cout << "\nPatient Added Successfully!\n";
            }
            catch(int)
            {
                cout << "\nInvalid Patient Type!";
                cout << "\nPlease Enter 1, 2 or 3.\n";
            }
            break;
        }
        // VIEW WAITING QUEUE
        case 2:
        {
            if(count == 0)
            {
                cout << "\nNo Patients In Queue.\n";
            }
            else
            {
                cout << "\n\n===== WAITING QUEUE =====\n";

                for(int i=0;i<count;i++)
                {
                    cout << "\nPosition: " << i + 1;

                    cout << "\nType: "
                         << queue[i]->getPatientType();

                    queue[i]->display();

                    cout << "\n--------------------------\n";
                }
            }
            break;
        }
        // CALL NEXT PATIENT
        case 3:
        {
            if(count == 0)
            {
                cout << "\nNo Patients Waiting.\n";
            }
            else
            {
                cout << "\nNEXT PATIENT\n";
                cout << "\nType: "
                     << queue[0]->getPatientType();
                queue[0]->display();
                delete queue[0];
                // Shift Array Left
                for(int i=0;i<count-1;i++)
                {
                    queue[i] = queue[i+1];
                }
                count--;

                cout << "\n\nPatient Sent To Doctor.\n";
            }
            break;
        }
        //=======================================
        // EXIT
```

```
262          //=======================================
263          case 4:
264          {
265              cout << "\nProgram Closed.\n";
266              break;
267          }
268          default:
269          {
270              cout << "\nInvalid Choice!\n";
271          }
272          }
273      }
274      while(choice != 4);
275      //=======================================
276      // MEMORY CLEANUP
277      //=======================================
278      for(int i=0;i<count;i++)
279      {
280          delete queue[i];
281      }
282      return 0;
283  }
```